

Auto-generating Quantized DNN Kernels using TVM

Dimitrije Pešić

1 Why Quantization Matters

ResNet50 has approximately 25 million parameters, occupying ~ 100 MB stored as float32. Per inference, it executes roughly 4 billion multiply-accumulate (MAC) operations. On edge hardware like Raspberry Pi 4, even a single inference can carry significant latency. Quantization addresses this by representing weights and activations as lower-precision numbers, trading numerical precision for reductions in memory, bandwidth, energy, and compute. It bridges the gap between models we can train and models we can deploy at scale.

Wu et al. (2020) frame int8 as a sweet spot for DNN quantization. The raw representation shrinks by $\sim 4\times$ (32-bit float to 8-bit integer); the empirical `.params` file ratio measured in this work is reported in Section 4.1. We also get proportional activation memory bandwidth reduction, and on hardware with int8 acceleration like Intel VNNI or ARM DOT, up to several-fold compute throughput improvements over fp32. Crucially, ResNet50 at int8 retains accuracy to within $\sim 0.1\%$ top-1 of the fp32 baseline (Wu et al. 2020), thanks to the asymmetric per-tensor scheme formalized by Jacob et al. (2018) and implemented as the canonical pattern in most modern inference toolchains.

Below int8, the precision/accuracy trade-off degrades sharply. Gholami et al. (2021) conclude that maintaining meaningful accuracy at sub-byte precisions requires either quantization-aware training or specialized network architectures.

The literature thus establishes int8 as the sweet spot: above it, the precision trade-off is essentially free; below it, accuracy degrades sharply without specialized training. This report investigates how TVM (Chen et al. 2018), a deep learning compiler, behaves across this boundary through its `relay.quantize` API. We compile and benchmark ResNet50 at fp32 and int8 on x86, analyze why the expected int8 speedup does not materialize with TVM’s defaults, test what happens at int4, int2, and int1, and cross-compile both fp32 and int8 to ARM for aarch64 execution under QEMU emulation. Throughout, we anchor TVM’s behavior to its source code rather than treating the framework as a black box.

2 TVM Compilation Pipeline

The workflow has three stages: (1) import the PyTorch model into TVM’s intermediate representation (Relay IR), (2) optionally apply quantization passes on the IR, and (3) lower the IR to machine code for a target backend.

2.1 Model Import

ResNet50 is loaded from `torchvision.models` with pretrained `IMAGENET1K_V2` weights. To produce a graph representation suitable for TVM, we trace the model with a dummy input of shape `[1,3,224,224]`:

```
scripted = torch.jit.trace(model, torch.randn(1, 3, 224, 224)).eval()
mod, params = relay.frontend.from_pytorch(
    scripted, [("input0", (1,3,224,224))])
```

This produces two objects from the converted TorchScript graph:

- `mod` – the computation graph in Relay IR
- `params` – the trained weights

2.2 Baseline Compilation (fp32)

We compile the Relay IR to x86 machine code by specifying a target. The target tells TVM which instruction set to emit, in this case LLVM’s x86 backend. For GPU compilation, this would be replaced with `cuda`.

The build runs inside `PassContext(opt_level=3)`, which enables TVM’s optimization passes: operator fusion, constant folding, and layout transformation. The last is why TVM’s logs show `conv2d_NCHWc` rather than `conv2d` – TVM rewrites the data layout from NCHW to a tiled format optimized for the x86 cache hierarchy.

```
target = tvm.target.Target("llvm", host="llvm")
with tvm.transform.PassContext(opt_level=3):
    lib = relay.build(mod, target=target, params=params)
```

The result is a compiled module containing generated machine code. We can inspect the compiled code at multiple levels. `lib.get_source("ll")` returns the LLVM IR (the intermediate representation just before machine code emission), and `lib.get_source("asm")` returns the generated assembly. For example, examining the LLVM IR reveals the `conv2d_NCHWc` kernel implementation with vectorized loop nests, confirming that TVM’s code generation produced real compute kernels rather than simply dispatching to an external library.

The compiled module is loaded into a `GraphModule` executor. We feed it a random fp32 input and benchmark over 100 runs.

```
m = graph_executor.GraphModule(lib["default"](dev))
m.set_input("input0", tvm.nd.array(img))
timing = m.benchmark(dev, number=100)
```

2.3 Int8 Quantization

TVM applies post-training quantization directly on the Relay IR – no retraining or calibration dataset is needed. The `quantize.qconfig` context manager controls all quantization parameters:

```
with quantize.qconfig(
    nbit_input=8, nbit_weight=8, nbit_activation=32,
    dtype_input="int8", dtype_weight="int8",
    dtype_activation="int32",
    calibrate_mode="global_scale", global_scale=8.0,
    skip_dense_layer=True, skip_conv_layers=[0],
):
    mod_int8 = quantize.quantize(mod, params=params)
```

`nbit_input` and `nbit_weight` control quantization precision – the parameter we vary later when testing sub-byte widths. The default values for every key here are defined in `QConfig._node_defaults` in `python/tvm/relay/quantize/quantize.py`, with a matching C++ definition in `src/relay/quantize/quantize.h`. This is why TVM “knows” to quantize to int8 even when `dtype_input` is not explicitly set: it is a baked-in framework default.

`calibrate_mode="global_scale"` uses a fixed scaling factor rather than running calibration data through the network; this is acceptable since we are benchmarking throughput, not accuracy. `skip_conv_layers=[0]` leaves the first convolution in fp32, a common practice since the input

layer has only 3 channels and quantizing it yields minimal savings. `skip_dense_layer=True` keeps the final classifier layer in fp32.

We verify that quantization took effect by printing the IR: `mod` shows float32 operations throughout, while `mod_int8` shows int8 casts and quantized convolutions. This confirms that `quantize.quantize` transformed the computation graph rather than merely annotating it. Internally, this transformation is implemented in three phases – *Annotate*, *Calibrate*, and *Realize* – with the operator-level rewriting performed in `src/relay/quantize/realize.cc` via per-op rules (`Conv2dRealize`, `AddRealize`, `MulRealize`, `ClipRealize`, etc.).

The quantized module is then compiled and benchmarked identically to the fp32 baseline:

```
with tvm.transform.PassContext(opt_level=3):
    lib_int8 = relay.build(mod_int8, target=target,
                           params=params)
```

3 Results

3.1 Benchmarking: fp32 vs int8

We benchmarked both models on x86 CPU (LLVM backend) over 100 inference runs each. All measurements were collected under the same conditions in a single session on a WSL2 host. The results:

Config	Mean (ms)	Median (ms)	Std (ms)
float32	166.59	166.28	1.21
int8 (signed)	379.92	373.71	9.99

We expected int8 to be at least as fast as fp32, and potentially several-fold faster on hardware with int8 acceleration. Instead, the int8 model is $2.28\times$ slower than fp32. Two factors explain this.

Lack of autotuning. TVM prints the warning “One or more operators have not been tuned. Please tune your model for better performance.” Without autotuning, TVM falls back to generic schedules for every operator. TVM’s autotuners (AutoTVM, Ansor) would search for optimal loop tiling, vectorization, and parallelization parameters for each operator shape on the target hardware. Without this step, the default schedules are naive and far from optimal.

Requantization overhead. The int8 computation graph is significantly more complex than fp32. Examining the `te_compiler` dispatch logs and the post-*Realize* IR, each convolution block in fp32 consists of three operations:

```
conv2d_NCHWc -> add -> relu
```

In int8, the same logical convolution becomes a chain of 13 operations:

```
conv2d_NCHWc -> add -> right_shift -> clip -> cast ->
cast -> multiply -> add -> relu -> add ->
right_shift -> clip -> cast
```

The extra operations (`right_shift`, `clip`, `cast`, `multiply`) implement the dequantization arithmetic: after computing in the int32 accumulator, the result must be rescaled (via a fixed-point multiplier or shift, depending on which path `MulAndDiv` in `realize.cc` selects), clipped to the int8 range, and cast back. This 13-op chain replaces every convolution in the network,

and additional requantization ops appear around the residual additions that join the ResNet bottleneck blocks.

Dispatch hypothesis. For int8 to be faster than fp32, TVM would need to dispatch the convolutions to a hardware-accelerated kernel. Tracing the dispatch logic, we find that `conv2d_NCHWc_strategy_cpu` (in `python/tvm/relay/op/strategy/x86.py`) chooses between two kernel implementations based on the helper `is_int8_hw_support` (defined in `python/tvm/topi/x86/conv2d_int8.py`). That helper imposes two conjoint requirements: (i) the dtypes must form an asymmetric pairing – `data_dtype == "uint8" and kernel_dtype == "int8"` – inherited from Intel’s VPDPBUSD intrinsic (visible in `python/tvm/topi/x86/tensor_intrin.py`), which operates on $\text{byte-unsigned} \times \text{signed-byte} \rightarrow \text{int32}$; and (ii) the compilation target must advertise the relevant ISA extension (`-mcpu=cascadelake` or similar enabling VNNI/AVX-512-VNNI). TVM’s default `qconfig` produces *signed* int8 for both data and weights, and our target string is just `"llvm"` (no `-mcpu` extension), so both conditions fail; dispatch falls through to the generic `conv2d_NCHWc.x86` kernel rather than the VNNI-accelerated `conv2d_NCHWc_int8.x86` path. The int8 graph thus pays the requantization overhead without gaining any compute throughput advantage. This dispatch hypothesis is tested empirically in Section 4.2, where the same int8 graph is cross-compiled to an ARM target with a different operator strategy.

In a separate experiment we tested the dtype-asymmetric configuration directly (`dtype_input="uint8", dtype_weight="int8"`). Run together in a single session for a controlled comparison, signed int8/int8 measured 396.10 ms and uint8/int8 measured 351.83 ms. The signed-int8 figure aligns with the 379.92 ms in the table above (Section 3.1) – the two runs are on the same hardware in separate sessions, so only the within-session comparison between signed and uint8 is fully controlled. The within-session result is that uint8/int8 is marginally faster ($\sim 11\%$) than signed int8/int8, and both remain far slower than the fp32 baseline (~ 166.59 ms). Inspecting the `te_compiler` dispatch log for the uint8/int8 run confirms that every convolution is still routed to the generic `conv2d_NCHWc.x86` kernel rather than the VNNI-accelerated `conv2d_NCHWc_int8.x86` path: with the target string left at the default `"llvm"`, condition (ii) of `is_int8_hw_support` (the `-mcpu` ISA-extension check) fails, so the asymmetric dtype pairing alone is not sufficient to unlock VNNI. The marginal speedup comes from cheaper requantization arithmetic on the uint8 path, not from a hardware-accelerated convolution. This is consistent with the absence of autotuning being the dominant factor on x86: even when the dtype pairing matches Intel’s intrinsic, an untuned generic schedule cannot beat the simpler fp32 graph.

3.2 Sub-byte Quantization: int4, int2, int1

The `qconfig` API accepts arbitrary bit widths through `nbit_input` and `nbit_weight`. We tested int4, int2, and int1 to determine which precisions TVM can handle end-to-end.

3.2.1 int4 (4-bit)

Quantization succeeded: `quantize.quantize()` returned a valid Relay module with int4 dtypes present in the IR (461 occurrences of “int4” in the IR text). However, `relay.build()` did not terminate: the process remained CPU-bound at $\sim 100\%$ on a single core for over 29 minutes inside `relay.build` before we terminated it. No exception, assertion, or progress message appeared after the standard “one or more operators have not been tuned” warning. The symptom – a non-terminating compile step with no diagnostic output – is the same shape as the int2 failure mode (Section 3.2.2) rather than a clean codegen-time check. Because the run was terminated externally, we cannot point to a specific assertion site; the closest documented TVM check for this case is `ICHECK_GE(value_dtype.bits(), 8)` in `CodeGenLLVM::BufferAccessHelper` (`src/target/llvm/codegen_llvm.cc`), which would fire if sub-byte buffer access reached LLVM codegen, but in this run the build stalled in an earlier pass before reaching that check.

3.2.2 int2 (2-bit)

Quantization aborted with a clean `InternalError` during `prerequisite_optimize` (`python/tvm/relay/quantize/quantize.py:329`). Python’s `faulthandler` stack dump (taken at 30s inside the call) shows the framework inside that helper, which transparently invokes a downstream `relay.build` – via `Interpreter::TIRToPackedFunc` and `driver.build` – in order to JIT-compile and execute small subgraphs for calibration. The build then propagates the `int2` storage dtype into TIR transforms, where `ArgBinder::BindDLTensor` (`src/tir/transforms/arg_binder.cc:277`) calls `GetVectorBytes` on the buffer dtype, and the runtime helper rejects it:

```
Check failed: data_bits % 8 == 0U (2 vs. 0):
Need to load/store by multiple of bytes
```

The error path is: `quantize.quantize` → `prerequisite_optimize` → `Interpreter::TIRToPackedFunc` → `driver.build` → `MakePackedAPI` → `ArgBinder::BindDLTensor` → `GetVectorBytes(int2)` → assertion failure. Two consequences are worth noting. First, the failure surface for `int2` is not in the Relay quantization pipeline at all but in the TIR-level buffer-binding pass, which encodes the architectural assumption that all storage dtypes are byte-multiples. Second, the calibration step transparently triggers a full compile of intermediate IR; sub-byte dtypes therefore have to survive every stage of *both* the Relay and TIR pipelines just to complete quantization, even before the user invokes `relay.build` explicitly.

3.2.3 int1 (1-bit)

Quantization succeeded, but `relay.build()` raised an `InternalError` with a clear stack trace. The failure occurs in the `SimplifyExpr` optimization pass (`src/relay/transforms/simplify_expr.cc`), in `CheckDataTypeMaxMinValue` which validates a clip op’s bounds against its target dtype. TVM treats `DataType::Int(1)` as effectively boolean with range `[0, 1]`, so constructing `IntImm(int1, -1)` for the quantization-generated clip’s lower bound fails the range assertion:

```
Check failed: (value == 0 || value == 1) is false:
ValueError: -1 exceeds range of int1
```

The error path is: `relay.build` → `OptimizeImpl` → `SimplifyExpr` → `SimplifyClipAndConsecutiveCast::Callback` → `CheckDataTypeMaxMinValue` → `tvm::min_value(int1)` → `IntImm(int1, -1)` → assertion failure.

3.2.4 Summary of Sub-byte Failures

Precision	Quantize	Build	Failure Location
int8	OK	OK	(works)
int4	OK	HANG	<code>relay.build</code> did not terminate; closest documented check is <code>ICHECK_GE(bits, 8)</code> in <code>CodeGenLLVM</code>
int2	FAIL	N/A	<code>ArgBinder::BindDLTensor</code> → <code>GetVectorBytes: data_bits % 8 == 0</code> (calibration’s hidden build)
int1	OK	FAIL	<code>SimplifyExpr: -1 exceeds int1 range</code>

These three failure modes confirm that TVM’s quantization framework (`Annotate` → `Calibrate` → `Realize`) is dtype-agnostic and accepts arbitrary `DataType::Int(N)` values without validation. The breakdown happens downstream when optimization or codegen passes encounter the resulting

IR. Each pass imposes its own dtype-specific assumptions, and there is no unified validation layer. The framework fails at varied stages, with varied symptoms ranging from clean exceptions to non-terminating loops, depending on which assumption is violated first.

Logical sub-byte mode. The split between quantization logic and downstream dtype handling can be confirmed directly. If we set `nbit_input=N` (with $N \in \{1, 2, 4\}$) but keep `dtype_input="int8"` and `dtype_weight="int8"` – a configuration we call *logical* sub-byte – the quantization pass reduces the effective range to N bits while keeping the IR’s storage dtype at int8. Build and execution succeed at every value of N because no sub-byte dtype ever flows downstream; the same int8 kernels run. The measured latencies for $N = 4$ and $N = 2$ stay within the int8 baseline’s range (419.08 ms and 403.71 ms respectively, against an int8 baseline of 379.92 ms from Section 3.1), confirming that the kernels are insensitive to a narrower input range when the storage dtype is unchanged. At $N = 1$, however, latency *collapses* to 9.81 ms – roughly an order of magnitude below the int8 baseline. This is not a kernel speedup: when the effective range is a single bit, the quantization pass produces a degenerate graph (most activations clip to a constant, weights collapse to $\{0\}$ or $\{0, \pm 1\}$ after clipping, and downstream constant-folding optimizations can eliminate most of the work). The shape of the latency curve across N thus tells us two distinct things: (a) $N \in \{2, 4\}$ confirms that the quantization pass itself has no objection to $nbit < 8$ and that what failed at native int4/int2/int1 is downstream dtype handling, not the quantization logic; (b) $N = 1$ exposes a separate phenomenon where the range reduction is so aggressive that algebraic simplification, not faster kernels, drives the speedup. The logical-mode measurements are produced by `experiments/02_bench_varied_bitwidth.py`.

3.3 Inspecting the Compiled Module

After compilation, TVM’s compiled module is not a black box – we can inspect the generated code at multiple levels of abstraction. Calling `lib.get_lib().get_source("ll")` returns the full LLVM IR, which reveals the vectorized loop nests TVM generated for each operator. For example, the main convolution kernel appears as a function with packed vector loads and fused multiply-add sequences, confirming that TVM produced real compute code rather than dispatching to an external BLAS library. At a lower level, `lib.get_lib().get_source("asm")` returns the target assembly; on x86, this shows SSE/AVX vector instructions corresponding to the LLVM IR’s vector operations.

```
# inspect generated code at LLVM IR level
llvm_ir = lib.get_lib().get_source("ll")
# inspect at assembly level
asm = lib.get_lib().get_source("asm")
# inspect the execution graph structure
graph = json.loads(lib.get_graph_json())
print(f"Number of nodes: {len(graph['nodes'])}")
```

The graph JSON structure exposes the execution order and operator names. For our fp32 ResNet50, this shows the fused `conv2d_NCHWc` operators alongside `add` and `relu` nodes, confirming that TVM’s operator fusion pass successfully merged element-wise operations into their preceding convolution. This inspection capability is important for diagnosing performance issues: when the int8 model was unexpectedly slow on x86, examining the graph JSON and LLVM IR confirmed that TVM did *not* emit VNNI intrinsics, corroborating the dispatch analysis of Section 3.1.

4 Cross-Compilation for ARM

4.1 Compiling for Raspberry Pi

To target the Raspberry Pi 4 (Cortex-A72), we change the compilation target string to specify the ARM instruction set:

```
target = tvm.target.Target(
    "llvm-device=arm_cpu_"
    "-mtriple=aarch64-linux-gnu_"
    "-mcpu=cortex-a72"
)
```

The target triple `aarch64-linux-gnu` instructs LLVM to emit 64-bit ARM code for Linux, and `mcpu=cortex-a72` enables microarchitecture-specific optimizations for the Pi 4’s CPU. Cortex-A72 is an ARMv8-A core with standard NEON SIMD; it does not include the dot-product instructions introduced in ARMv8.2+.

Since we are cross-compiling (building ARM binaries on an x86 host), we use `aarch64-linux-gnu-gcc` as the cross-compiler when emitting the shared library:

```
from tvm.contrib import cc as _cc

def cross_cc(output, files, options=None,
             cc="aarch64-linux-gnu-gcc"):
    return _cc.create_shared(output, files,
                             options, cc=cc)

lib.export_library("resnet50_fp32_arm.so",
                   fcompile=cross_cc)
```

Both the fp32 and int8 models are exported as three files each:

- `.so` – the compiled shared library containing operator kernels
- `.json` – the graph structure (execution order, operator names, shapes)
- `.params` – the serialized weights

This produces six files total. The aarch64 ELF format of each `.so` was verified with the `file` utility:

```
$ file resnet50_fp32_arm.so
resnet50_fp32_arm.so: ELF 64-bit LSB shared object,
ARM aarch64, version 1 (SYSV), dynamically linked,
with debug_info, not stripped
```

The int8 `.params` file is 30.4 MB versus 191.5 MB for fp32 (a $6.30\times$ ratio in the serialized files, larger than the raw $4\times$ ratio cited in Section 1 because TVM’s serialization metadata is included in both file sizes), confirming the storage benefit of int8 quantization independent of any compute-side speedup.

4.2 QEMU Emulation

Choice of machine type. The natural reading of the assignment – “run on a Raspberry Pi emulator under QEMU” – points to QEMU’s family of dedicated Raspberry Pi machines (`raspi0`, `raspi1ap`, `raspi2b`, `raspi3ap`, `raspi3b`). In the QEMU version available on our system (`qemu-system-aarch64 6.2.0`), this family stops at `raspi3b`; the Raspberry Pi 4 machine (`raspi4b`) was

only added in later QEMU releases (8.2+). Since our cross-compile target is `-mcpu=cortex-a72` (the Raspberry Pi 4’s CPU, Section 4.1), choosing `raspi3b` would force execution on an emulated Cortex-A53 instead, introducing an A53/A72 microarchitecture mismatch between the LLVM scheduler and the CPU running the code. The generic ARM `virt` machine, by contrast, accepts an explicit `-cpu` flag and lets us emulate Cortex-A72 directly. We therefore chose `virt` with `-cpu cortex-a72` as the closest available match to a Raspberry Pi 4 environment: a faithful A72 ISA execution of the same binary that would run on physical Pi 4 hardware, even though the surrounding machine is not the literal Pi board. All ARMv8-A cores share the NEON SIMD ISA used by TVM’s `int8` kernels, so the qualitative findings of this section depend on the emulated CPU, not on board-specific peripherals.

To execute the cross-compiled binaries without physical Raspberry Pi hardware, we set up full-system emulation under `qemu-system-aarch64` using the generic ARM `virt` machine type with `-cpu cortex-a72`, four virtual CPUs, and 4 GB of RAM. The guest is Ubuntu 22.04 Server (aarch64 cloud image) booted from UEFI firmware (`QEMU_EFI.fd`), with networking exposed via user-mode `hostfwd` forwarding host port 2222 to guest port 22 so that we can SSH into the VM directly. Inside the guest we installed Python, NumPy, and built the TVM v0.15.0 runtime from source (`make runtime` in the TVM build directory; LLVM is not required for a runtime-only build). The six exported model files were copied into the guest over SCP and a small benchmark script was run via SSH.

Caveat on emulated vs native A72 performance. The emulated CPU model (`-cpu cortex-a72`) matches the compilation target (`-mcpu=cortex-a72`) exactly, so LLVM’s instruction selection is aligned with the CPU TVM expects – there is no A53/A72 microarchitecture mismatch. However, QEMU is a software CPU emulator (TCG, not KVM, since the host is x86) and does not preserve A72’s pipeline depth, issue width, branch predictor, or cache hierarchy. Absolute QEMU latencies therefore overestimate native A72 execution by a large factor; the *ratio* between `fp32` and `int8` is informative because both configurations incur the same emulation cost on the same emulated CPU.

A small benchmark script inside the VM loads each artifact triple via `tvm.runtime.load_module`, `graph_executor.create`, and `load_params`, runs a single warmup iteration, and measures latency using TVM’s `benchmark()` API with `number=2`, `repeat=3` (kept low because of QEMU overhead).

Config	Mean (ms)	Median (ms)
float32 (aarch64, QEMU)	9550.5	9603.8
int8 (aarch64, QEMU)	4619.2	4693.5

Table 1: ResNet50 inference latency under QEMU full-system emulation (`virt` machine, emulated Cortex-A72, Ubuntu 22.04 aarch64 guest, 4 GB RAM). Each measurement is the mean of three repeats of two inferences each, preceded by a single warmup run. QEMU is software-emulating the CPU on an x86 host, so absolute values overestimate native A72 latency by a large factor; the ratio between `fp32` and `int8` is informative because both configurations incur the same emulation cost on the same emulated CPU.

Two observations stand out. First, both models produced sensible classification outputs (`fp32 argmax=21`, `int8 argmax=812`, with output ranges spanning several units around zero). The different `argmax` values are expected: with a random (non-ImageNet) input and coarse global-scale calibration, the `int8` approximation diverges from `fp32` in its top prediction while still producing a plausible class-score distribution, confirming that the cross-compiled aarch64 binaries execute correctly. Second, and more interestingly, under aarch64 emulation the `int8` model runs $2.07\times$ *faster* than `fp32` – the opposite of what we measured on x86, where `int8` was

2.28 $\times$ *slower*.

This inversion is direct empirical support for the dispatch hypothesis developed in Section 3.1. On x86, TVM’s optimized int8 conv kernel requires both an asymmetric `uint8  $\times$  int8` dtype pairing and a target that advertises VNNI; neither condition is met by our default `qconfig` and “`llvm`” target, so dispatch falls through to a generic kernel. On aarch64, the int8 kernel paths in TVM’s ARM operator strategy (under `python/tvm/topi/arm_cpu/`) appear not to gate on the same `uint8/int8` asymmetric pairing; the signed int8 inputs and weights produced by the default `qconfig` flow through an int8-specific schedule that benefits from reduced data movement and from NEON’s vectorized integer multiply-accumulate. We did not trace the ARM dispatch through source to the same depth as the x86 path; the inverted latency ratio measured here is the empirical evidence for this hypothesis. The expected quantization speedup materializes – even under QEMU’s substantial emulation overhead. The bottleneck on x86 was therefore the specific interaction between TVM’s default `qconfig` and the platform’s dispatch logic, not int8 quantization itself.

4.3 Performance Comparison

Figure 1 summarizes the latency results across both platforms. On x86, int8 is 2.28 $\times$ *slower* than fp32 due to the dispatch and requantization issues discussed in Section 3.1. On aarch64 (under QEMU), the relationship inverts: int8 is 2.07 $\times$ *faster*, demonstrating that the expected quantization speedup materializes when the target’s operator strategy accepts signed int8 directly.

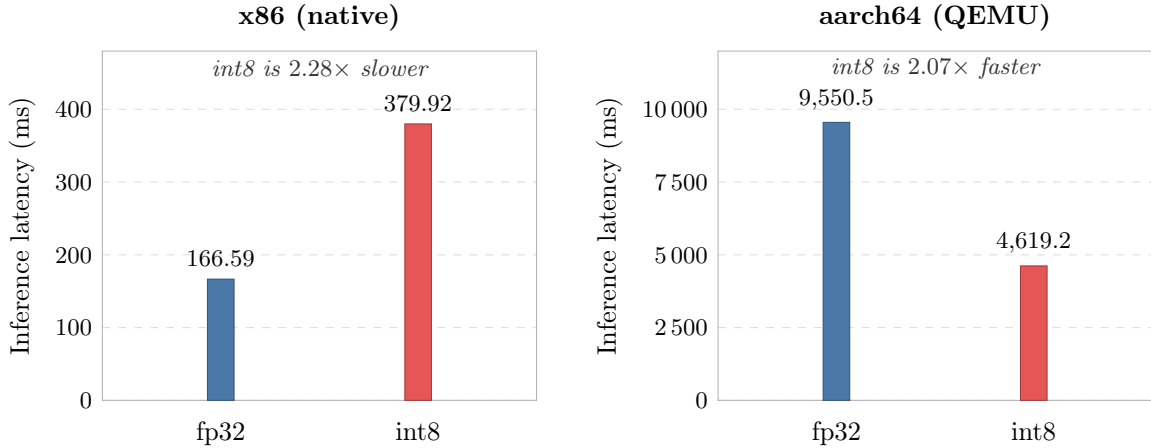


Figure 1: ResNet50 inference latency on x86 (native) and aarch64 (QEMU emulated), shown with independent y-axis scales per platform so the within-platform ratio between fp32 and int8 is visible. The relationship inverts: int8 is 2.28 $\times$ *slower* on x86 (dispatch miss; Section 3.1) but 2.07 $\times$ *faster* on aarch64 (proper int8 kernel selection). Absolute values are not comparable across platforms because of QEMU emulation overhead.

5 Conclusion

This report treated TVM not as a black box but as an inspectable compiler: we read its IR, traced its dispatch logic in source, examined the generated LLVM IR and assembly, and used cross-platform execution to empirically validate a hypothesis derived from code reading. The central finding is that quantization performance is not a property of the precision alone – it depends on whether the compiler’s dispatch logic, the target’s ISA constraints, and the `qconfig`’s dtype choices align. When they do (aarch64, signed int8), the textbook $\sim 2\times$ speedup appears;

when they don't (x86, signed int8 vs VNNI's uint8 requirement), quantization *hurts* due to requantization overhead without compensating throughput gains.

Sub-byte quantization revealed a different lesson: TVM's frontend is permissive (it will quantize to any bit width) but the downstream pipeline – optimization passes, codegen – is not. The three distinct failure modes at int4, int2, and int1 show that end-to-end dtype support requires coordinated assumptions across every compiler stage, not just the quantization pass itself.

For production deployment, three concrete steps would unlock the expected x86 speedup: (i) adopting a uint8-input/int8-weight qconfig, (ii) compiling against a target that advertises VNNI (e.g. `-mcpu=cascadelake`) so `is_int8_hw_support` succeeds and the VNNI-accelerated convolution is dispatched, and (iii) running AutoTVM or Ansor to replace the naive default schedules with hardware-tuned ones. (i) alone is not sufficient: we verified empirically that uint8 input with the bare "llvm" target still falls through to the generic kernel. On ARM, targeting ARMv8.2+ cores with dot-product extensions would provide an additional throughput tier beyond the NEON-based speedup we observed. The weight compression measured here (30.4 MB int8 vs 191.5 MB fp32 in the serialized `.params` files, a $6.30\times$ ratio) remains independently valuable for memory-constrained edge deployment regardless of compute-side dispatch.

Scope of measurement: untuned baselines, by design. AutoTVM/MetaSchedule tuning was not performed for two reasons. Tuning ResNet50 at the granularity required to draw meaningful conclusions takes several hours per target $\times$ configuration; at the matrix of configurations explored here, this would have exceeded ten hours of compute. More importantly, tuning under QEMU emulation produces a meaningless timing signal for the ARM target because the measurements that AutoTVM would use to score candidate schedules reflect the emulator's overhead rather than native hardware behavior. The reported numbers therefore represent default-schedule baselines – the natural reference point for analyzing how quantization interacts with the compiler's out-of-the-box behavior, before hardware-specific tuning enters the picture. The two findings of this report (x86 int8 slowdown from dispatch miss; aarch64 int8 speedup from proper kernel selection) concern the structure of the dispatch logic and the IR, not schedule quality, and hold independently of whether tuning is applied on top.

References

- [1] Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., & Krishnamurthy, A. (2018). TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of OSDI '18*.
- [2] Wu, H., Judd, P., Zhang, X., Isaev, M., & Micikevicius, P. (2020). Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. *arXiv:2004.09602*.
- [3] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2021). A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv:2103.13630*.
- [4] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of CVPR '18*, pp. 2704–2713.